AI & Programming Hackathon – 30-Hour Challenge

Duration: 30 hours

Participants: 3rd & 4th-year Computer Science & Engineering Students

Objective: Solve real-world AI problems by implementing practical applications in Python with APIs, NLP, and AI tools.

General Instructions

- Each team selects one task to implement.
- Use **Python** with libraries like HuggingFace, FAISS/Chroma/Qdrant, FastAPI or Flask.
- Provide clean, structured code, with README, sample API requests, and example outputs.
- **Do not hallucinate outputs;** all answers, citations, or evidence must come from the dataset only.
- Teams must handle missing/Invalid inputs gracefully.
- Include logging and error handling in your implementation.

Tasks Overview

Task No	Task Name	Key Objective	Difficulty
1	Document Q&A Bot	Answer user questions from documents with citations	Medium
2	Customer Support Ticket Triage &	Classify tickets, estimate urgency, draft grounded replies	Medium
3	Duplicate Defect Finder & Bug Report	Detect duplicate defects, cluster reports, improve quality	Medium
4	Meeting Minutes & Action Items Generator	Summarize meetings, extract decisions, action items, risks	Medium
5	Product Review Insights API	Extract aspects, sentiment, pros & cons with evidence	Medium

Evaluation Criteria

Criterion	Weight
Correctness & Functionality	**40%**
AI/ML Implementation Quality	**30%**
API Design & Engineering	**20%**
Documentation & README	**10%**

Do's	Don'ts
• **Ground** outputs in dataset evidence • **Validate** API input/output • Provide README with setup & examples • Handle invalid inputs gracefully	• **No hallucination** or fake sources • Don't expose sensitive data • Avoid overly complex models

Hackathon Duration: 30 Hours

AI & Programming Hackathon – 30-Hour Challenge

Duration: 30 hours
Participants: 3rd & 4th-year Computer Science & Engineering students
Objective: Solve real-world AI problems by implementing practical applications in Python with APIs, NLP, and AI tools.

General Instructions

- Each team selects **one task** to implement.
- Use **Python** with libraries like HuggingFace Transformers, FAISS/Chroma/Qdrant, FastAPI or Flask.
- Provide **clean, structured code**, with **README, sample API requests, and example outputs**.
- **Do not hallucinate outputs**; all answers, citations, or evidence must come from the dataset only.
- Handle missing or invalid inputs gracefully.
- Include **logging and error handling** in your implementation.

Tasks Overview

Task No	Task Name	Key Objective	Difficulty
1	Document Q&A Bot (RAG)	Answer user questions from documents with citations	Medium
2	Customer Support Ticket Triage & Draft Reply	Classify tickets, estimate urgency, draft grounded replies	Medium
3	Duplicate Defect Finder & Bug Report Enhancer	Detect duplicate defects, cluster reports, improve report quality	Medium
4	Meeting Minutes & Action Items Generator	Summarize meetings, extract decisions, action items, risks	Medium
5	Product Review Insights API	Extract aspects, sentiment, pros & cons, with evidence snippets	Medium

---

Task Details

Task 1: Document Q&A Bot using RAG

Problem Statement:
Build a system that answers questions from a folder of documents (policies, FAQs, manuals, SOPs). The system must: - Search the documents
- Retrieve relevant content
- Answer using **only retrieved content**
- Return **citations** (document + snippet)

Mandatory: If answer not found, respond:
> "I could not find this in the provided documents. Can you share the relevant document?"

Inputs: - Folder `docs/` (.txt, .md, .pdf)
- User question (API POST request)

Outputs: - `answer` (short paragraph)
- `sources` (document + snippet + score)
- `confidence`: "high" | "medium" | "low"

Mandatory Components: - Document ingestion + chunking
- Embeddings + vector store
- Top-k retrieval
- Answer generation using retrieved chunks
- Citation generation (no fabricated sources)

Datasets: - Wikipedia Plain Text 2020
- CUAD Contract Dataset
- Wikipedia 2023 Dump

The precision and coverage of the response based on the input documents matters

---

Task 2: Customer Support Ticket Triage & Draft Reply

Problem Statement:
Build a system to classify support tickets, estimate urgency, and draft replies using a Knowledge Base (KB). The system must: - Classify ticket category (Refund, Login, Delivery, Billing, Account, Other)
- Estimate urgency (High/Medium/Low)
- Retrieve KB snippets
- Draft short reply grounded in KB

Mandatory: Do **not guess**; ask clarification if KB lacks information.

Inputs: - Ticket text (subject + description)
- Optional metadata (channel, timestamp)
- KB folder (`.txt/.md/.pdf`)

Outputs: - `category`
- `urgency`
- `summary`
- `draft_reply`
- `sources` (KB citations)
- `confidence`

Mandatory Components: - Ticket preprocessing
- Category & urgency logic (rules or ML)
- KB ingestion + embeddings
- Top-k retrieval
- Grounded reply generation

Datasets: - Customer Support Ticket Dataset
- Multilingual Customer Support Tickets
- HuggingFace Tickets Dataset

Task 3: Duplicate Defect Finder & Bug Report Enhancer

Problem Statement:
- Detect duplicate defect reports
- Assign cluster IDs
- Suggest missing fields
- Generate improved defect summary

Mandatory: Only mark duplicates if similarity above threshold; show top matches.

Inputs: - Defect dataset (CSV/JSON): `defect_id`, `title`, `description`, `steps`, `expected`, `actual`, `environment`, `logs`
- New defect report (JSON)

Outputs: - `decision`: "duplicate" | "possible_duplicate" | "new_defect"
- `top_matches` (up to 5)
- `cluster_id`
- `improved_report` (title, summary, missing fields)
- `confidence`

Mandatory Components: - Text normalization & embeddings
- Vector database + similarity search
- Clustering (DBSCAN or nearest cluster)
- Missing field detection
- Safe summary generation

Datasets: - Bugzilla Bug Reports
- GitBugs
- Jira Issue Reports

Task 4: Meeting Minutes & Action Items Generator

Problem Statement:
- Generate meeting summary
- Extract decisions, action items (owner + due date), risks/questions

Mandatory: If owner/due date missing, return `null` or ask clarification.

Inputs: - Transcript (`.txt` or JSON)
- Optional metadata (date, attendees)

Outputs: - `meeting_summary` (5–8 bullets)
- `decisions`
- `action_items` (task, owner, due date, evidence)
- `risks_open_questions`
- `confidence`

Mandatory Components: - Transcript cleanup
- Structured extraction
- Evidence snippet capture
- JSON validation

Datasets: - AMI Meeting Corpus
- OpenSLR AMI Dataset
- HuggingFace AMI Dataset

> Tip: Limit to 5–20 meetings for the hackathon.

Task 5: Product Review Insights API

Problem Statement:
- Extract top aspects/topics
- Compute sentiment per aspect
- Generate pros & cons with supporting evidence
- Handle "not enough data" safely

Mandatory: Do **not invent** pros/cons without evidence.

Inputs: - Reviews dataset (CSV): `product_id`, `review_text`, `rating`, `review_date`
- API request with `product_id`

Outputs: - `top_aspects` (aspect, sentiment, score)
- `pros` (with evidence)
- `cons` (with evidence)
- `summary`
- `confidence`

Mandatory Components: - Text preprocessing
- Aspect extraction (keywords, clustering, or LLM)

- Aspect sentiment analysis
- Evidence selection
- API validation

Datasets: - Amazon Product Reviews
- Amazon Electronics Reviews
- Datafiniti Amazon Reviews

Evaluation Criteria

Criterion	Weight
Correctness & functionality	40%
AI/ML implementation quality	30%
API design & engineering	20%
Documentation & README	10%

Do's

- Ground outputs in dataset evidence
- Validate API input/output
- Provide README with setup, run instructions, and example API calls
- Handle missing/invalid inputs gracefully

Don'ts

- Do not hallucinate outputs or fabricate citations
- Do not expose sensitive data
- Avoid overly complex models; clarity and correctness are key

---

End of Document